

Instituto Tecnológico y de Estudios Superiores de Monterrey



Avance 0. Propuesta de proyecto y firma de convenios Proyecto integrador (Gpo 10)

Implementación de MLOps para versionado, monitoreo y decisión de
retraining en un sistema de recomendación de precios B2B

Y-PIE MLOPS

Integrantes del equipo:

~

Mario Javier Soriano Aguilera | A01384282

Emanuel Flores Martínez | A01796497

David Alberto Serrano Garcia | A01795935

Fecha de entrega: 3 de mayo de 2026

Índice

Índice	2
1. Antecedentes	4
2. Entendimiento del negocio.....	4
2.1 Formulación del problema	4
2.2 Contexto	5
2.3 Objetivo general	6
2.4 Objetivos específicos.....	6
2.5 Preguntas clave	6
2.6 Involucrados	7
3. Entendimiento de los datos.....	8
3.1 Descripción de los datos	8
3.2 Contexto de los datos.....	8
3.3 Técnica de ML y enfoque MLOps	9
3.4 Variables de entrada	9
3.5 Variables de salida	10
3.6 Consideraciones de calidad, seguridad, trazabilidad y uso de Azure	10
3.7 Arquitectura de infraestructura, IaC y CI/CD extendido.....	11
Separación entre lógica MLOps e infraestructura	11
Repositorio MLOps: lógica de pricing	12
Repositorio de infraestructura.....	12
Infraestructura como código	13
Recursos mínimos recomendados para el MVP	13
CI/CD para infraestructura	14
Infra CI	14
Infra CD.....	15
Integración entre MLOps e infraestructura	15
Relación con trazabilidad y observabilidad	15
Beneficios esperados.....	16
3.8 Gobernanza de infraestructura y gestión de entornos.....	16
Gestión de entornos.....	17
Escenario mínimo recomendado para MVP	17
Escenario extendido	17
Promoción de cambios	18

Gobernanza de recursos cloud.....	18
Convención de nombrado.....	19
Etiquetado de recursos	20
Control de accesos y RBAC (Por revisar ya que puede agregar complejidad)	20
Gestión de secretos	21
Gestión de costos	21
Presupuestos y alertas	21
Estrategia de sizing.....	21
Apagado y limpieza de recursos.....	22
Seguridad y control de datos	22
Lifecycle de infraestructura	23
Observabilidad de infraestructura.....	23
Relación entre observabilidad MLOps e infraestructura	24
Consideraciones de alcance.....	24
4. Cronograma de trabajo	25
5. Alcance inicial y trabajo futuro	25
5.1 Alcance inicial.....	25
5.2 Fuera del alcance inicial.....	26
6. Bibliografía	26
7. Anexos	26
Anexo A. Estructura propuesta de repositorio	26
Anexo B. Tabla propuesta model_run_log.....	27
Anexo C. Tabla propuesta model_drift_log.....	27
Anexo D. Regla inicial de semáforo	28
Anexo E. Nota de confidencialidad	28
Anexo F. Uso responsable de la cuenta de Azure.....	28
Anexo G. Estructura propuesta de repositorio de infraestructura.....	29
Anexo H. Convenio de confidencialidad firmado	30

1. Antecedentes

El proyecto se desarrolla en el contexto de una organización dedicada a la manufactura y comercialización de componentes electrónicos, donde el proceso de pricing tiene un impacto directo en los ingresos, márgenes, competitividad comercial y toma de decisiones de negocio. En este tipo de entorno B2B, las decisiones de precio no dependen únicamente del valor histórico de venta, sino también de múltiples variables como producto, cliente, distribuidor, región comercial, segmento, cantidad, margen, costos, reglas de negocio y comportamiento histórico de negociación.

Actualmente, el área de Pricing Business Intelligence trabaja con modelos y procesos analíticos que permiten generar recomendaciones de precios a partir de información histórica. Estas recomendaciones pueden incluir precios objetivo, precios piso y precios de inicio para negociación. Además, el proceso considera diferentes niveles de granularidad, como número de parte, región comercial, segmento de distribuidor, grupo de negocio y distribuidor o cliente.

Aunque los modelos de recomendación de precios se consideran estables para el alcance actual del proyecto, existe una necesidad importante de fortalecer la trazabilidad, reproducibilidad, monitoreo y gobierno del ciclo de vida del sistema. En particular, se requiere saber qué versión de código, configuración, dataset, lógica de percentiles/modelado y outputs fueron utilizados en cada corrida del sistema.

El proyecto, por lo tanto, no se enfoca principalmente en desarrollar un modelo nuevo desde cero, sino en implementar una capa MLOps que permita operar el sistema de pricing de forma más confiable, auditable y mantenible. Esta capa considerará elementos como versionado, CI/CD mínimo viable, detección de drift, observabilidad, snapshots de outputs y reglas de decisión para determinar cuándo se debe revisar, recalibrar o reentrenar el modelo.

Desde la perspectiva de CRISP-ML(Q), el proyecto se ubica principalmente en las fases de operación, monitoreo y mantenimiento del ciclo de vida de machine learning. Esto es relevante porque, incluso cuando un modelo es estable, sus datos de entrada, distribuciones históricas, reglas de negocio y outputs recomendados pueden cambiar con el tiempo. Por esta razón, la confiabilidad del sistema no depende únicamente del modelo, sino también de la capacidad para monitorear y controlar su comportamiento en operación.

La actividad solicita construir una propuesta de proyecto integrador considerando antecedentes, entendimiento del negocio, entendimiento de los datos, cronograma si aplica, bibliografía y anexos, tomando como base la metodología CRISP-ML. En ese sentido, este documento adapta dicha metodología al enfoque específico de MLOps, priorizando la operación confiable del sistema de recomendación de precios.

2. Entendimiento del negocio

2.1 Formulación del problema

El sistema de recomendación de precios cuenta con modelos considerados estables para generar recomendaciones basadas en información histórica de precios, cantidades, márgenes, costos y segmentaciones comerciales. Sin embargo, el proceso actual requiere

una capa formal de MLOps que permita asegurar trazabilidad, control de versiones, monitoreo de cambios y toma de decisiones sobre mantenimiento del modelo.

El problema principal no consiste en seleccionar un nuevo algoritmo ni en reconstruir completamente el modelo de pricing, sino en responder preguntas operativas críticas como:

- ¿Qué versión de código generó una recomendación específica?
- ¿Qué configuración o reglas estaban activas en cada corrida?
- ¿Qué dataset o snapshot fue utilizado?
- ¿Qué lógica de percentiles/modelado se aplicó?
- ¿Qué outputs se generaron?
- ¿Las distribuciones de precio, cantidad o recomendaciones han cambiado de forma relevante?
- ¿El sistema debe mantenerse igual, revisarse o reentrenarse?

Sin una capa MLOps, el sistema puede enfrentar riesgos como baja reproducibilidad, dificultad para auditar resultados, poca visibilidad sobre cambios en datos, falta de criterios objetivos para retraining y dependencia de validaciones manuales.

Por ello, el proyecto busca diseñar e implementar una primera versión funcional de una capa MLOps para el sistema de pricing, enfocada en versionado, CI/CD, drift detection, observabilidad y reglas de decisión para retraining.

2.2 Contexto

En un entorno de pricing B2B, las recomendaciones de precio tienen impacto directo en decisiones comerciales, márgenes, ingresos y competitividad. Una recomendación incorrecta puede generar pérdida de margen, pérdida de ventas, baja confianza de los usuarios de negocio o inconsistencias en negociaciones con clientes y distribuidores.

Además, los modelos de pricing pueden volverse menos confiables no necesariamente porque el algoritmo falle, sino porque cambian las condiciones de negocio. Por ejemplo, pueden cambiar las distribuciones de precios, cantidades vendidas, mix de productos, costos, márgenes, regiones, segmentos comerciales o comportamiento de clientes.

En este sentido, MLOps permite conectar el modelo con una operación controlada. Para este proyecto, las prácticas MLOps más relevantes son:

- Versionado: control de cambios en código, configuración, datasets, lógica de percentiles/modelado y outputs.
- CI/CD mínimo viable: validación automática de cambios antes de usarlos en una corrida controlada.
- Drift Detection: detección de cambios en precio, cantidad y outputs recomendados.
- Retraining Decision: reglas de negocio y métricas para decidir si se mantiene el modelo, se revisa o se reentrena.
- Observabilidad: registros y snapshots que permitan auditar cada corrida y entender el comportamiento del sistema.

El valor del proyecto está en fortalecer la confianza operacional del sistema de pricing, permitiendo que las recomendaciones sean trazables, explicables y monitoreables.

Aunque el proyecto no se enfoca principalmente en desarrollar un nuevo modelo de machine learning, sí se alinea con CRISP-ML(Q), ya que atiende fases fundamentales del ciclo de vida de ML: operación, monitoreo, mantenimiento, evaluación continua y control de calidad. El sistema de pricing cuenta con modelos base considerados estables, por lo que el mayor riesgo actual no está en la selección del algoritmo, sino en asegurar que sus recomendaciones sean reproducibles, trazables y monitoreables conforme cambian los datos, configuraciones y condiciones de negocio.

2.3 Objetivo general

Diseñar e implementar una capa MLOps mínima viable para un sistema estable de recomendación de precios B2B, con el propósito de asegurar el versionado de código, configuración, datasets, lógica de percentiles/modelado y outputs, así como establecer mecanismos de CI/CD, detección de drift, observabilidad y decisión de retraining.

2.4 Objetivos específicos

- Diseñar una estructura de versionado para código, configuración, datasets utilizados, lógica de percentiles/modelado y outputs generados.
- Implementar un flujo CI/CD mínimo viable que permita validar cambios en código, configuración y lógica antes de ser utilizados en una corrida del sistema.
- Definir una tabla de trazabilidad de ejecuciones mediante un registro tipo `model_run_log`.
- Crear snapshots de outputs del modelo para conservar evidencia de las recomendaciones generadas en cada corrida.
- Definir un baseline histórico contra el cual puedan compararse nuevas corridas del sistema.
- Implementar métricas de drift detection para monitorear cambios en precio, cantidad y outputs recomendados.
- Diseñar una lógica de decisión basada en semáforos green/yellow/red para clasificar el estado del sistema.
- Incorporar una lógica de impacto ponderado por revenue para priorizar los cambios más relevantes desde el punto de vista de negocio.
- Documentar el flujo MLOps propuesto, sus componentes, limitaciones y posibles extensiones futuras.
- Utilizar de manera responsable una cuenta de Azure asignada al equipo para pruebas, almacenamiento, ejecución controlada de componentes MLOps y documentación de la solución.

2.5 Preguntas clave

- ¿Qué versión de código, configuración, dataset y lógica generó cada corrida del sistema de pricing?
- ¿Cómo se puede asegurar que una recomendación de precio sea reproducible?
- ¿Qué información mínima debe registrarse en cada ejecución del sistema?
- ¿Qué variables deben monitorearse para detectar drift en un sistema de recomendación de precios?
- ¿Cómo se puede detectar drift en precio, cantidad y outputs recomendados?

- ¿Qué cambios deben clasificarse como green, yellow o red?
- ¿Cuándo un cambio en los datos requiere solo revisión y cuándo requiere retraining?
- ¿Cómo ponderar el impacto del drift según revenue o importancia comercial?
- ¿Cómo puede integrarse CI/CD para validar cambios en la lógica antes de ejecutar el sistema?
- ¿Qué componentes deben quedar fuera del alcance inicial y considerarse como trabajo futuro?
- ¿Cómo se puede utilizar Azure de manera controlada para evitar costos innecesarios durante el desarrollo del proyecto?

2.6 Involucrados

Involucrado	Rol dentro del proyecto	Nivel de participación	Tipo de involucramiento
Equipo de Pricing BI / Data Science	Diseño de la capa MLOps, análisis de datos, implementación de versionado, drift y observabilidad	Alto	Técnico y analítico
Product Managers	Validación de criterios de negocio, interpretación de recomendaciones y revisión de alertas	Alto	Negocio
Equipo comercial / ventas	Usuario indirecto de las recomendaciones de precio	Medio	Consulta y retroalimentación
Área de IT / Data Engineering	Apoyo en acceso a datos, repositorios, bases de datos, permisos e infraestructura	Medio	Técnico
Sponsor del proyecto	Autoriza acceso a datos y valida relevancia organizacional	Alto	Dirección y autorización
Profesor / asesor académico	Evalúa metodología, documentación y avance del proyecto	Medio	Académico
Cuenta de Azure asignada al equipo	Ambiente cloud para desarrollo, pruebas, almacenamiento, ejecución de pipelines y posible implementación de componentes MLOps	Alto	Infraestructura y operación controlada

Tabla 1. Involucrados principales del proyecto y nivel de participación.

Para el desarrollo del proyecto se brindará una cuenta de Azure al equipo, la cual será utilizada como ambiente de trabajo para construir, probar y documentar los componentes de la solución MLOps. El uso de esta cuenta deberá realizarse con cautela, procurando optimizar recursos, evitar gastos innecesarios y monitorear el consumo para no agotar el presupuesto disponible.

3. Entendimiento de los datos

3.1 Descripción de los datos

Los datos utilizados en el proyecto corresponden a información histórica relacionada con transacciones comerciales, precios, cantidades, productos, clientes, distribuidores, costos, márgenes y segmentaciones de negocio. Estos datos permiten alimentar el sistema de recomendación de precios y, al mismo tiempo, monitorear la estabilidad de sus resultados.

El proyecto considera que el sistema de pricing ya cuenta con modelos base estables. Por ello, los datos no solo se analizan para entrenamiento, sino también para trazabilidad, validación continua, comparación contra baselines y detección de drift.

Las principales categorías de datos son:

Categoría de datos	Descripción	Uso en el proyecto
Datos de producto	Número de parte, jerarquías de producto y grupo de negocio	Segmentación y granularidad de recomendación
Datos de cliente/distribuidor	Cliente, distribuidor, parent number y segmento	Análisis por canal y recomendación por segmento
Datos de región	VP Area u otra clasificación geográfica/comercial	Análisis regional de comportamiento
Datos de precio	Precio histórico, precio de venta o precio de reventa	Modelado, percentiles y drift de precio
Datos de cantidad	Cantidad vendida u ordenada	Percentiles, demanda y drift de cantidad
Datos de rentabilidad	Costos, márgenes y revenue	Validación de negocio e impacto ponderado
Outputs del modelo	Precio recomendado, precio target, floor y start	Observabilidad y drift de recomendaciones
Configuración	Parámetros, reglas y thresholds	Reproducibilidad y control de cambios
Logs de ejecución	Fecha, versión de código, dataset y estado	Auditoría y trazabilidad

Tabla 2. Tipos de datos considerados en el proyecto MLOps.

3.2 Contexto de los datos

Los datos representan operaciones comerciales históricas y resultados generados por el sistema de recomendación de precios. En términos generales, cada registro puede estar asociado con un producto, cliente o distribuidor, segmento, región, fecha, precio, cantidad, costo y revenue.

Para el enfoque MLOps, los datos cumplen cuatro funciones principales:

1. Entrada del sistema de pricing: alimentan la lógica de percentiles, modelado y generación de recomendaciones.
2. Base de comparación histórica: permiten crear un baseline contra el cual se comparan futuras corridas.

3. Evidencia de cada ejecución: permiten documentar qué datos fueron usados y qué resultados se generaron.
4. Monitoreo de estabilidad: permiten detectar drift en variables críticas y outputs recomendados.

Este enfoque es importante porque el riesgo principal del proyecto no es únicamente que el modelo tenga bajo desempeño, sino que el entorno de datos cambie con el tiempo y las recomendaciones pierdan coherencia sin que el equipo lo detecte oportunamente.

3.3 Técnica de ML y enfoque MLOps

El sistema de pricing puede incluir técnicas de aprendizaje supervisado, principalmente modelos de regresión, elasticidad, lógica basada en percentiles y optimización de precios. Sin embargo, el presente proyecto no tiene como objetivo principal desarrollar un nuevo modelo ni comparar múltiples algoritmos.

El proyecto se enfoca en aplicar prácticas MLOps alrededor de un sistema existente. Por ello, las técnicas principales estarán orientadas a monitoreo estadístico, comparación de distribuciones, versionado, validación automatizada y reglas de decisión.

Componente	Técnica o enfoque
Modelo base de pricing	Regresión, elasticidad, percentiles y optimización
Versionado	Git/GitHub, versiones de configuración, datasets y outputs
CI/CD	Validación automática de código, configuración, pruebas y schemas
Drift de precio	Comparación de percentiles, cambios relativos, PSI o JSD
Drift de cantidad	Comparación de distribución y percentiles de cantidad
Drift de outputs	Comparación de recomendaciones actuales vs. baseline
Retraining decision	Semáforo green/yellow/red con impacto ponderado
Observabilidad	model_run_log, snapshots y drift logs
Azure	Ambiente cloud controlado para pruebas, almacenamiento y ejecución de componentes

Tabla 3. Relación entre componentes técnicos y enfoque MLOps.

De esta manera, el proyecto sigue la lógica de CRISP-ML(Q), pero se concentra principalmente en las fases posteriores al desarrollo inicial del modelo: evaluación continua, operación, monitoreo y mantenimiento.

3.4 Variables de entrada

Las variables de entrada pueden agruparse de la siguiente manera:

Grupo	Variables ejemplo
Identificadores	KPN, VP Area, Disty Segment, Parent Number, Business Group
Producto	Product Level 1, Product Level 2, Product Level 3, Product Level 4, Product Level 5, Product Level 6

Cliente/distribuidor	Customer ID, Customer Number, Distributor Parent Number
Transacción	Invoice Date, Date Period, Revenue, Quantity
Precio	Precio histórico, precio de reventa, precio facturado
Rentabilidad	Costo actual, margen, margen distribuidor
Configuración	Config version, thresholds de drift, reglas activas
Ejecución	Run ID, timestamp, git commit, environment
Infraestructura	Recursos de Azure utilizados, ubicación de almacenamiento, estado de ejecución

Tabla 4. Variables de entrada consideradas para el sistema y la capa MLOps.

3.5 Variables de salida

Las salidas principales se dividen en salidas de negocio y salidas MLOps.

Tipo de salida	Variable o artefacto	Descripción
Negocio	Recommended target price	Precio objetivo recomendado
Negocio	Recommended floor price	Precio mínimo sugerido
Negocio	Recommended start price	Precio inicial sugerido para negociación
Negocio	Pricing strategy	Estrategia asociada al precio
MLOps	model_run_log	Registro de cada corrida
MLOps	model_output_snapshot	Snapshot de recomendaciones generadas
MLOps	model_drift_log	Registro de métricas de drift
MLOps	Drift status	Estado green/yellow/red
MLOps	Retraining decision	No action, review, recalibrate o retrain
MLOps / Cloud	Registro de uso de recursos	Evidencia del uso controlado de servicios cloud

Tabla 5. Salidas esperadas del sistema de pricing y de la capa MLOps.

3.6 Consideraciones de calidad, seguridad, trazabilidad y uso de Azure

Debido a que los datos de pricing, clientes, costos y márgenes pueden ser sensibles, el proyecto considera medidas de confidencialidad. En caso de ser necesario, se utilizarán datos anonimizados, agregados, simulados o enmascarados que mantengan una estructura similar al proceso real, pero sin exponer información confidencial.

Las principales consideraciones son:

1. Confidencialidad: evitar exposición de clientes, precios reales, márgenes sensibles o información estratégica.
2. Calidad de datos: validar valores nulos, duplicados, cantidades negativas, precios inválidos y registros inconsistentes.
3. Granularidad: definir claramente el nivel de análisis y recomendación.

4. Reproducibilidad: asociar cada output con código, configuración, dataset y lógica usada.
5. Trazabilidad: mantener registros de ejecución y snapshots históricos.
6. Validación de negocio: revisar que las recomendaciones sean coherentes con reglas de pricing, márgenes y comportamiento histórico.
7. Control de cambios: usar CI/CD mínimo viable para evitar que cambios no validados entren al flujo principal.
8. Uso responsable de infraestructura cloud: debido a que se proporcionará una cuenta de Azure para operar la solución, se deberán utilizar los recursos con cautela, evitando despliegues innecesarios, servicios sobredimensionados o procesos activos sin supervisión que puedan consumir el presupuesto disponible.

3.7 Arquitectura de infraestructura, IaC y CI/CD extendido

Con el propósito de fortalecer la capa MLOps más allá del versionado, monitoreo, trazabilidad y control del modelo, se propone incorporar una capa de infraestructura que permita gestionar de forma estructurada los recursos cloud utilizados por el sistema de recomendación de precios B2B.

Esta capa se basa en el uso de Infrastructure as Code (IaC), así como en la separación explícita entre la lógica de machine learning, la lógica de pricing y la infraestructura sobre la cual se ejecutan los procesos. De esta manera, la infraestructura deja de ser un conjunto de configuraciones manuales y pasa a formar parte del sistema como un componente versionable, auditable, reproducible y sujeto a validaciones.

El enfoque propuesto permite mejorar la reproducibilidad, trazabilidad, escalabilidad y control operativo del sistema, alineándose con prácticas modernas de MLOps y DevOps. Esto resulta especialmente relevante en el contexto del proyecto, ya que el sistema de pricing no solo requiere generar recomendaciones de precio, sino también asegurar que cada ejecución pueda asociarse con una versión específica de código, configuración, dataset, lógica de percentiles/modelado, outputs generados y recursos cloud utilizados.

Separación entre lógica MLOps e infraestructura

Se propone dividir la solución en dos repositorios principales, con el objetivo de mantener una separación clara de responsabilidades entre los componentes funcionales del sistema y los componentes de infraestructura.

El primer repositorio corresponde al repositorio MLOps del sistema de pricing. Este repositorio contiene la lógica funcional del proyecto, incluyendo preparación de datos, lógica de percentiles, componentes de modelado, validaciones, drift detection, reglas de decisión, generación de outputs, snapshots, registros de ejecución y configuraciones versionadas.

El segundo repositorio corresponde al repositorio de infraestructura. Este repositorio contiene la definición de los recursos cloud utilizados por el sistema, la configuración de ambientes, los módulos reutilizables de infraestructura, los pipelines de CI/CD para infraestructura y los scripts necesarios para facilitar el despliegue controlado de recursos.

Esta separación permite que los cambios en la lógica del modelo no dependan directamente de cambios en la infraestructura y viceversa. Asimismo, facilita el mantenimiento, la auditoría y la evolución del proyecto en fases posteriores.

Repositorio MLOps: lógica de pricing

El repositorio MLOps se enfoca en los componentes funcionales y analíticos del sistema. Su propósito principal es controlar la lógica que permite preparar los datos, ejecutar el sistema de pricing, generar recomendaciones, validar resultados y monitorear la estabilidad del sistema.

Este repositorio incluirá, entre otros elementos:

- Código de preparación y transformación de datos.
- Lógica de percentiles, reglas de pricing y componentes de modelado.
- Implementación de drift detection para precio, cantidad y outputs recomendados.
- Validaciones de calidad de datos, schemas y consistencia de outputs.
- Pipelines de ejecución del sistema de pricing.
- Configuraciones versionadas del modelo, thresholds, reglas y parámetros.
- Pruebas unitarias o funcionales mínimas.
- Generación de registros como `model_run_log` y `model_drift_log`.
- Generación y almacenamiento de snapshots de outputs.

El objetivo de este repositorio es asegurar que la lógica de pricing pueda ser versionada, validada y reproducida de manera consistente.

Repositorio de infraestructura

El repositorio de infraestructura se enfoca en la definición, creación, actualización y eliminación controlada de los recursos cloud requeridos por el sistema. Este repositorio permitirá gestionar la infraestructura mediante código, evitando configuraciones manuales difíciles de auditar o reproducir.

Este repositorio incluirá, entre otros elementos:

- Definición de recursos cloud mediante IaC.

- Configuración de ambientes, como staging, test, dev o prod, según el alcance y presupuesto disponible.
- Módulos reutilizables para recursos comunes.
- Variables y archivos de configuración por ambiente.
- Pipelines de validación y despliegue de infraestructura.
- Scripts auxiliares para inicialización, despliegue o limpieza de recursos.
- Documentación técnica sobre recursos creados, propósito y responsables.

La existencia de un repositorio independiente de infraestructura permite que los recursos utilizados por el sistema de pricing sean tratados como parte formal del proyecto y no como componentes externos no controlados.

Infraestructura como código

Componente de infraestructura	Propósito dentro del proyecto
Storage Account o Data Lake	Almacenar datasets, snapshots, outputs, logs y artefactos del sistema
Contenedores de almacenamiento	Separar entradas, salidas, baselines, logs y artefactos
Recursos de cómputo	Ejecutar pipelines, pruebas o procesos controlados del sistema
Servicios de orquestación	Coordinar la ejecución de procesos de pricing, validación y monitoreo
Key Vault	Gestionar secretos, credenciales, tokens y cadenas de conexión
Servicios de monitoreo	Registrar eventos, errores, métricas de ejecución y uso de recursos
Resource Groups	Agrupar recursos por proyecto, ambiente o propósito
Roles y permisos	Controlar accesos mediante RBAC
Presupuestos y alertas	Monitorear el consumo y evitar costos innecesarios

Tabla 6. Componentes de infraestructura considerados para la extensión MLOps.

La selección final de recursos dependerá del alcance permitido por la cuenta de Azure asignada al equipo, así como de las restricciones de presupuesto del proyecto. Para una primera versión, se priorizará una arquitectura mínima viable que permita demostrar el valor de la capa MLOps sin generar complejidad o costos innecesarios.

Recursos mínimos recomendados para el MVP

Para el alcance inicial del proyecto, no es necesario implementar una arquitectura cloud compleja. Se recomienda iniciar con recursos mínimos que permitan validar la trazabilidad, almacenamiento, ejecución controlada y monitoreo básico del sistema.

Los recursos mínimos sugeridos son:

Recurso	Uso esperado
Resource Group	Agrupar todos los recursos del proyecto
Storage Account	Almacenar datasets, outputs, snapshots y logs
Contenedores separados	Organizar entradas, salidas, baselines y registros
Key Vault	Proteger secretos y cadenas de conexión
Compute básico o servicio equivalente	Ejecutar pruebas, pipelines o procesos programados
Azure Monitor o Log Analytics	Registrar eventos técnicos y apoyar la observabilidad
Budget Alerts	Controlar consumo de la cuenta de Azure

Tabla 7. Recursos mínimos sugeridos para una arquitectura cloud controlada.

Este enfoque permite mantener una solución ligera, controlada y alineada con el carácter académico y de mínima viabilidad del proyecto, sin perder la posibilidad de escalar hacia una arquitectura más robusta en fases posteriores.

CI/CD para infraestructura

Además del CI/CD mínimo viable planteado para la lógica MLOps, se propone incorporar flujos específicos para validar y desplegar infraestructura. Estos flujos permitirán asegurar que los cambios en recursos cloud sean revisados, validados y aplicados de forma controlada.

Se definen dos flujos principales: Infra CI e Infra CD.

Infra CI

El flujo de Infra CI se ejecuta antes de aplicar cambios sobre la infraestructura. Su objetivo es validar que el código de infraestructura sea consistente, esté correctamente definido y no genere cambios inesperados.

Este flujo puede incluir:

- Validación sintáctica del código Terraform o Bicep.
- Revisión de formato del código de infraestructura.
- Validación de variables requeridas.
- Verificación de configuración por ambiente.
- Generación de un plan de despliegue.
- Revisión del impacto esperado de los cambios.
- Bloqueo de cambios si existen errores en la definición.

El resultado esperado de Infra CI es un plan de despliegue que permita visualizar qué recursos serán creados, modificados o eliminados antes de aplicar los cambios.

Infra CD

El flujo de Infra CD se encarga de aplicar los cambios validados sobre el ambiente correspondiente. Este flujo debe ejecutarse de forma controlada, especialmente si los recursos pueden generar costos o afectar componentes utilizados por el sistema.

Este flujo puede incluir:

- Aplicación del plan de despliegue aprobado.
- Creación o actualización de recursos cloud.
- Registro del estado de despliegue.
- Publicación de outputs de infraestructura.
- Notificación de éxito o error.
- Actualización de referencias utilizadas por el repositorio MLOps.

Para ambientes de mayor criticidad, como producción, se recomienda incorporar aprobaciones manuales antes de aplicar cambios. En el contexto del proyecto, esta práctica puede definirse conceptualmente aunque no se implemente de forma completa.

Integración entre MLOps e infraestructura

La integración entre el repositorio MLOps y el repositorio de infraestructura se realizará mediante configuraciones compartidas, referencias a recursos provisionados y servicios centralizados.

Esta integración puede lograrse mediante:

- Variables de entorno.
- Archivos de configuración por ambiente.
- Outputs generados por Terraform.
- Referencias a rutas de almacenamiento.
- Cadenas de conexión protegidas en Key Vault.
- Identificadores de recursos cloud.
- Configuraciones de pipelines.
- Parámetros versionados dentro del repositorio MLOps.

Por ejemplo, el repositorio de infraestructura puede crear una cuenta de almacenamiento y exponer como output el nombre del recurso o las rutas de contenedores. Posteriormente, el repositorio MLOps puede utilizar esas referencias para almacenar snapshots, consultar baselines o registrar logs de ejecución.

Este mecanismo permite mantener una separación clara entre la creación de recursos y el uso funcional de esos recursos por parte del sistema de pricing.

Relación con trazabilidad y observabilidad

La capa de infraestructura no reemplaza los componentes de observabilidad ya definidos en el proyecto, como `model_run_log`, `model_output_snapshot` y `model_drift_log`. En cambio, los complementa.

Mientras los logs MLOps permiten responder preguntas sobre la ejecución del sistema, los datos utilizados, los outputs generados y las métricas de drift, la observabilidad de

infraestructura permite responder preguntas sobre el comportamiento técnico de la plataforma.

Algunas preguntas que puede responder la capa de infraestructura son:

- ¿Qué recurso ejecutó una corrida específica?
- ¿En qué ambiente se ejecutó el pipeline?
- ¿Qué cuenta de almacenamiento fue utilizada?
- ¿Existieron errores técnicos durante la ejecución?
- ¿Cuánto tiempo tomó la ejecución?
- ¿Qué recursos estuvieron activos?
- ¿Hubo consumo anormal de cómputo o almacenamiento?
- ¿Qué cambios de infraestructura ocurrieron antes de una corrida?

Esta relación fortalece la trazabilidad integral del sistema, ya que permite conectar las recomendaciones generadas con el código, configuración, datos, lógica e infraestructura utilizados.

Beneficios esperados

La incorporación de una capa de infraestructura gestionada como código aporta beneficios relevantes para el proyecto:

- Mayor reproducibilidad de ambientes.
- Reducción de configuraciones manuales.
- Mejor control sobre recursos cloud.
- Separación clara entre lógica de pricing e infraestructura.
- Mayor trazabilidad sobre cambios técnicos.
- Posibilidad de validar cambios antes de desplegarlos.
- Control de costos mediante recursos mínimos y alertas.
- Mejor preparación para una futura operación productiva.
- Mayor alineación con prácticas modernas de MLOps y DevOps.

En conjunto, esta arquitectura extendida permite que el sistema de pricing evolucione desde una capa MLOps mínima viable hacia una plataforma más robusta, auditable y escalable.

3.8 Gobernanza de infraestructura y gestión de entornos

Como extensión de la arquitectura MLOps propuesta, se identifica la necesidad de incorporar una capa de gobernanza de infraestructura que permita gestionar de forma controlada los recursos cloud, los accesos, los costos, la seguridad y la evolución del sistema a lo largo del tiempo.

Este enfoque resulta especialmente relevante en sistemas de pricing, donde la sensibilidad de los datos, el impacto en negocio y la necesidad de trazabilidad requieren un mayor nivel de control operativo. En este tipo de sistemas, una ejecución incorrecta, un acceso no

autorizado o una configuración inconsistente puede afectar la confianza en las recomendaciones, la interpretación de resultados o el uso responsable de recursos cloud.

La gobernanza de infraestructura no implica necesariamente implementar una arquitectura empresarial compleja desde el inicio. Para el alcance del proyecto, se plantea como una definición conceptual y práctica inicial, orientada a controlar riesgos, facilitar la trazabilidad y preparar el sistema para una posible evolución futura.

Gestión de entornos

Se propone la definición de ambientes de trabajo separados, con el objetivo de distinguir actividades de desarrollo, validación y futura operación del sistema.

La cantidad de ambientes no se define de forma rígida, ya que dependerá de la complejidad del sistema, el presupuesto disponible y las restricciones de la cuenta de Azure asignada al equipo.

Se consideran dos escenarios posibles.

Escenario mínimo recomendado para MVP

Para una primera versión del proyecto, puede ser suficiente trabajar con un ambiente de validación o staging. Este ambiente permitiría ejecutar pruebas funcionales y técnicas sin necesidad de crear una arquitectura completa con múltiples entornos.

El escenario mínimo podría incluir:

Ambiente	Propósito
staging/test	Validar lógica de pricing, pipelines, logs, snapshots, drift detection e integración con recursos cloud
producción futura	Ambiente conceptual para una futura ejecución con impacto operativo o de negocio

Tabla 8. Escenario mínimo de ambientes para el proyecto.

Este enfoque permite demostrar la viabilidad técnica del sistema sin multiplicar recursos ni incrementar innecesariamente el consumo de Azure.

Escenario extendido

En una fase posterior, si el sistema evoluciona hacia una operación más robusta, se puede considerar una separación más completa de ambientes.

El escenario extendido podría incluir:

Ambiente	Propósito
dev	Experimentación, desarrollo inicial y pruebas de bajo riesgo
test/staging	Validación controlada de código, configuración, datos e infraestructura

prod	Operación real del sistema con impacto en negocio
------	---

Tabla 9. Escenario extendido de ambientes para una evolución futura.

Cada ambiente debería contar con recursos independientes o lógicamente separados, definidos mediante infraestructura como código. Esto permitiría reducir riesgos, evitar interferencias entre pruebas y operación, y facilitar la promoción controlada de cambios.

Para el alcance actual, se recomienda mantener una arquitectura ligera, priorizando uno o dos ambientes como máximo, y dejando documentado el diseño para escalar a más ambientes en el futuro.

Promoción de cambios

Se propone un flujo controlado de promoción de cambios entre ambientes. Este flujo puede adaptarse según la cantidad de entornos implementados.

En un escenario mínimo, el flujo podría ser:

staging → prod

En un escenario extendido, el flujo podría ser:

dev → test/staging → prod

La promoción de cambios debe considerar tanto componentes de machine learning como componentes de infraestructura. Esto significa que no solo se deben validar cambios en código de pricing o drift detection, sino también cambios en configuraciones, recursos cloud, variables de ambiente y conexiones.

El flujo de promoción debe incluir:

- Validaciones automáticas mediante CI/CD.
- Revisión de configuración antes de ejecución.
- Validación de schemas de datos y outputs.
- Revisión del plan de cambios de infraestructura.
- Aprobaciones manuales antes de ambientes críticos.
- Registro de cambios aplicados.
- Evidencia de resultados generados.

En el contexto de pricing, las aprobaciones manuales son especialmente importantes porque las recomendaciones pueden tener impacto directo en márgenes, ingresos, competitividad y decisiones comerciales.

Gobernanza de recursos cloud

Para asegurar consistencia y control sobre los recursos cloud, se propone definir lineamientos básicos de gobernanza. Estos lineamientos no requieren necesariamente una implementación empresarial completa, pero sí permiten establecer una base ordenada para el proyecto.

Los principales lineamientos recomendados son:

Lineamiento	Descripción
Convención de nombrado	Estandarizar nombres de recursos para identificar proyecto, ambiente, tipo de recurso y propósito
Etiquetado obligatorio	Asignar metadatos como proyecto, ambiente, responsable, centro de costos y fecha de creación
Separación por ambiente	Evitar mezclar recursos de desarrollo, validación y producción
Control de propietarios	Documentar quién creó cada recurso y para qué se utiliza
Revisión periódica	Identificar recursos inactivos, duplicados o innecesarios
Uso de IaC	Evitar recursos creados manualmente sin registro
Eliminación controlada	Borrar recursos temporales cuando ya no sean necesarios

Tabla 10. Lineamientos básicos de gobernanza de infraestructura.

La implementación completa de estas prácticas puede incrementar la complejidad del proyecto. Sin embargo, su definición conceptual se considera un valor agregado importante, ya que alinea la solución con estándares utilizados en entornos profesionales de MLOps y cloud.

Convención de nombrado

Se recomienda definir una convención de nombrado para los recursos cloud. Esto facilita la identificación de recursos, reduce ambigüedades y permite auditar el uso de infraestructura de manera más sencilla.

Una posible estructura de nombres podría ser:

<proyecto>-<ambiente>-<tipo-recurso>-<propósito>

Ejemplos conceptuales:

Recurso	Ejemplo de nombre
Resource Group	pricing-mlops-stg-rg
Storage Account	pricingmlopsstgsa
Key Vault	pricing-mlops-stg-kv
Log Analytics	pricing-mlops-stg-logs
Compute	pricing-mlops-stg-compute

Tabla 11. Ejemplo conceptual de convención de nombrado.

La convención final deberá adaptarse a las restricciones de nombres de cada servicio de Azure, ya que algunos recursos no permiten guiones, mayúsculas o nombres largos.

Etiquetado de recursos

El etiquetado o tagging permite agregar metadatos a los recursos cloud. Esto facilita la trazabilidad, el control de costos, la identificación de responsables y la clasificación por ambiente.

Se recomienda considerar las siguientes etiquetas mínimas:

Etiqueta	Descripción
project	Nombre del proyecto
environment	Ambiente asociado, por ejemplo dev, staging o prod
owner	Responsable técnico o equipo responsable
cost_center	Centro de costo o referencia académica/proyecto
purpose	Propósito del recurso
created_by	Usuario o proceso que creó el recurso
lifecycle	Temporal, permanente o experimental

Tabla 12. Etiquetas recomendadas para recursos cloud.

Estas etiquetas permiten identificar rápidamente recursos que pueden eliminarse, recursos asociados a pruebas y recursos que forman parte de la operación principal del sistema.

Control de accesos y RBAC (Por revisar ya que puede agregar complejidad)

Debido a que el sistema utiliza datos relacionados con pricing, clientes, costos, márgenes y revenue, se recomienda aplicar controles de acceso basados en roles. El objetivo es que cada usuario o componente tenga únicamente los permisos necesarios para cumplir su función.

Se propone considerar roles diferenciados para:

Rol	Permisos esperados
Data Science / Pricing BI	Acceso a datasets anonimizados, ejecución de notebooks o pipelines y consulta de resultados
Data Engineering / IT	Gestión de recursos técnicos, conexiones, permisos e integración
Product Managers	Consulta de reportes, alertas y criterios de negocio
Administrador cloud	Gestión controlada de recursos, costos y políticas
Pipeline automatizado	Permisos limitados para ejecutar procesos y escribir logs o outputs

Tabla 13. Roles conceptuales para control de accesos.

Para el alcance del proyecto, puede bastar con documentar estos roles y aplicar permisos básicos. En una fase futura, podrían implementarse controles más estrictos mediante Azure RBAC, identidades administradas y políticas de acceso específicas.

Gestión de secretos

La gestión de secretos es un componente relevante para evitar que credenciales, tokens o cadenas de conexión queden expuestas en el código fuente o archivos de configuración.

Se recomienda utilizar un servicio como Azure Key Vault para almacenar:

- Cadenas de conexión.
- Tokens de acceso.
- Credenciales de servicios.
- Llaves de almacenamiento.
- Parámetros sensibles.
- Secretos utilizados por pipelines.

El repositorio MLOps no debería contener secretos en texto plano. En su lugar, los pipelines o procesos de ejecución deberían recuperar los secretos desde un servicio centralizado y seguro.

Esta práctica reduce el riesgo de exposición accidental de información sensible y mejora la seguridad del sistema.

Gestión de costos

La gestión de costos es una consideración central del proyecto, debido a que se utilizará una cuenta de Azure asignada al equipo. El objetivo es demostrar la viabilidad técnica de la solución sin generar consumo innecesario ni utilizar recursos sobredimensionados.

Se propone complementar el uso responsable de la infraestructura con mecanismos formales de control de costos.

Presupuestos y alertas

Se recomienda configurar presupuestos y alertas automáticas para monitorear el consumo de la cuenta de Azure.

Estas prácticas pueden incluir:

- Definición de un límite mensual de consumo.
- Alertas al alcanzar ciertos porcentajes del presupuesto.
- Revisión periódica del costo acumulado.
- Identificación de recursos con mayor consumo.
- Notificación al equipo ante incrementos inesperados.

Estrategia de sizing

Para ambientes de desarrollo o staging, se recomienda utilizar recursos de bajo costo o configuraciones mínimas.

La estrategia de sizing debe considerar:

- Uso de cómputo pequeño para pruebas.
- Ejecución bajo demanda en lugar de recursos siempre activos.
- Uso de datasets reducidos, anonimizados o simulados.
- Escalamiento únicamente cuando sea necesario.
- Evitar servicios premium si no son indispensables para el MVP.

Apagado y limpieza de recursos

Se recomienda definir prácticas de apagado automático o eliminación de recursos temporales.

Estas prácticas pueden incluir:

- Apagar máquinas virtuales o cómputo cuando no estén en uso.
- Eliminar recursos creados para pruebas puntuales.
- Evitar procesos permanentes sin supervisión.
- Documentar recursos activos y responsables.
- Revisar periódicamente recursos sin uso.

Estas acciones permiten mantener controlado el presupuesto y refuerzan el uso responsable de la cuenta de Azure.

Seguridad y control de datos

Dado el carácter sensible de los datos de pricing, se recomienda incorporar controles adicionales de seguridad y privacidad.

Los controles sugeridos son:

Control	Propósito
Uso de datos anonimizados	Evitar exposición de clientes, precios reales o información estratégica
Restricción de accesos	Limitar datasets y outputs según rol
Key Vault	Proteger secretos, credenciales y conexiones
Registro de accesos	Mantener evidencia de consultas o ejecuciones relevantes
Separación de ambientes	Evitar uso de datos sensibles en ambientes de prueba
Enmascaramiento de datos	Reducir exposición de información confidencial
Validaciones de calidad	Detectar inconsistencias antes de ejecutar el sistema

Tabla 14. Controles sugeridos de seguridad y datos.

En ambientes no productivos, se recomienda utilizar datos anonimizados, agregados, simulados o enmascarados. Esta práctica es consistente con las consideraciones de confidencialidad ya definidas en el proyecto y permite realizar pruebas sin exponer información sensible.

Lifecycle de infraestructura

Se propone considerar el ciclo de vida completo de los recursos de infraestructura, desde su creación hasta su eliminación. Esto permite evitar acumulación de recursos innecesarios y mejorar la eficiencia operativa.

El ciclo de vida recomendado incluye:

Etapas	Descripción
Creación	Despliegue automatizado mediante IaC
Configuración	Parametrización de recursos, permisos y conexiones
Uso	Ejecución controlada de pipelines, almacenamiento de outputs y generación de logs
Monitoreo	Observación de errores, consumo y comportamiento técnico
Actualización	Cambios versionados mediante CI/CD
Revisión	Evaluación periódica de recursos activos y costos
Eliminación	Limpieza de recursos temporales o inactivos

Tabla 15. Ciclo de vida propuesto para infraestructura cloud.

Este enfoque ayuda a mantener la infraestructura alineada con las necesidades reales del proyecto y evita que se acumulen recursos sin propósito definido.

Observabilidad de infraestructura

Además del monitoreo del modelo y de los outputs, se propone incorporar una capa de observabilidad sobre la infraestructura. Esta capa permite registrar el comportamiento técnico de los recursos y complementar la trazabilidad funcional del sistema.

La observabilidad de infraestructura puede incluir:

- Monitoreo de ejecución de pipelines.
- Registro de errores técnicos.
- Métricas de uso de cómputo.
- Métricas de almacenamiento.
- Tiempos de ejecución.
- Estado de recursos.
- Fallas de conexión.
- Eventos de despliegue.
- Consumo asociado a recursos cloud.

Esta capa puede implementarse mediante herramientas como Azure Monitor, Log Analytics o registros generados por los propios pipelines.

La observabilidad de infraestructura complementa la observabilidad MLOps ya definida en el proyecto, particularmente los registros `model_run_log`, `model_output_snapshot` y `model_drift_log`.

Relación entre observabilidad MLOps e infraestructura

La observabilidad del sistema puede dividirse en dos niveles complementarios:

Nivel de observabilidad	Preguntas que responde
Observabilidad MLOps	Qué código, dataset, configuración, lógica y outputs se usaron en cada corrida
Observabilidad de infraestructura	Qué recursos ejecutaron el proceso, qué errores técnicos ocurrieron y qué consumo se generó

Tabla 16. Relación entre observabilidad MLOps e infraestructura.

Al combinar ambos niveles, el sistema puede responder preguntas operativas más completas. Por ejemplo, si una corrida falla, se puede revisar tanto el estado del pipeline y del recurso cloud como la configuración, dataset y versión de código utilizada.

Consideraciones de alcance

La implementación completa de una capa avanzada de gobernanza puede considerarse fuera del alcance inicial del proyecto. Sin embargo, su diseño conceptual se incluye debido a su relevancia para la evolución del sistema hacia un entorno productivo robusto, seguro y escalable.

Para el alcance actual, se recomienda priorizar:

- Definición de repositorio de infraestructura.
- Uso de IaC para recursos básicos.
- Documentación de ambientes.
- Control inicial de costos.
- Uso responsable de Azure.
- Gestión básica de secretos.
- Observabilidad mínima de infraestructura.
- Separación conceptual entre lógica MLOps e infraestructura.

En fases posteriores, podrían incorporarse prácticas más avanzadas como políticas empresariales, múltiples regiones, alta disponibilidad, automatización completa de despliegues productivos, monitoreo avanzado y gobierno centralizado.

4. Cronograma de trabajo

Aunque el proyecto se alinea con CRISP-ML(Q), su naturaleza está más orientada a MLOps que al desarrollo de un modelo desde cero. Por esta razón, se incluye un cronograma específico para planificar y dar seguimiento a las actividades del curso.

El alcance se delimita a una primera versión funcional de una capa MLOps. No se contempla en esta fase un retraining automático completo, monitoreo en tiempo real, feature store o despliegue automatizado a producción. Estos elementos se consideran extensiones futuras.

Semana	Actividad principal	Entregable esperado	Relación con MLOps
Semana 1	Definir alcance, antecedentes y objetivos del proyecto	Propuesta del proyecto	Entendimiento del negocio
Semana 2	Documentar flujo actual del sistema de pricing	Diagrama del proceso actual y puntos de control	Diagnóstico operativo
Semana 3	Diseñar estructura de repositorio y versionado	Repositorio base con carpetas para código, SQL, configs y tests	Versionado
Semana 4	Definir CI/CD mínimo viable	Workflow de validación de código, configuración y schemas	CI/CD
Semana 5	Diseñar model_run_log y registro de versiones	Tabla de trazabilidad de ejecuciones	Observabilidad
Semana 6	Crear baseline histórico de comparación	Snapshot base de precio, cantidad y outputs	Monitoreo
Semana 7	Implementar drift detection de precio y cantidad	Métricas iniciales de drift	Drift Detection
Semana 8	Implementar drift de outputs recomendados y semáforo	Green/yellow/red y reglas de acción	Retraining Decision
Semana 9	Integrar logs, snapshots y reporte de validación	Evidencia de corridas y resultados	Observabilidad
Semana 10	Documentar arquitectura final y preparar presentación	Reporte final y presentación ejecutiva	Comunicación y cierre

Tabla 17. Cronograma de trabajo propuesto para el proyecto MLOps.

5. Alcance inicial y trabajo futuro

5.1 Alcance inicial

El proyecto incluirá:

- Versionado de código, configuración, dataset usado, lógica de percentiles/modelado y outputs.
- CI/CD mínimo viable para validar cambios.
- Registro de corridas mediante model_run_log.

- Creación de baseline histórico.
- Drift detection para precio, cantidad y outputs recomendados.
- Semáforo green/yellow/red.
- Reglas de decisión para revisión, recalibración o retraining.
- Observabilidad mediante logs y snapshots.
- Uso controlado de una cuenta de Azure asignada para pruebas, almacenamiento, ejecución de componentes MLOps y documentación de la solución.

5.2 Fuera del alcance inicial

Se consideran fuera del alcance inicial:

- Retraining completamente automático.
- Monitoreo en tiempo real.
- Feature store.
- Despliegue automático a producción.
- CI/CD empresarial completo con múltiples ambientes.
- Kubernetes o infraestructura cloud avanzada.
- Dashboard avanzado de producción.
- Uso de servicios cloud de alto costo o configuraciones sobredimensionadas que no sean necesarias para el MVP del proyecto.

Estos elementos podrán abordarse en fases posteriores, una vez que la capa MLOps mínima viable haya sido validada.

6. Bibliografía

Hernández-Sampieri, R., & Mendoza, C. (2023). Metodología de la investigación: Las rutas de la investigación cuantitativa, cualitativa y mixta. McGraw-Hill.

Visengeriyeva, L., Kammer, A., Bär, I., Kniesz, A., & Plöd, M. (2023). CRISP-ML(Q): The ML lifecycle process. MLOps. INNOQ. <https://ml-ops.org/content/crisp-ml>

7. Anexos

Anexo A. Estructura propuesta de repositorio

```
pricing-mlops/
|
|-- src/
|   |-- data_preparation/
|   |-- percentile_logic/
|   |-- pricing_model/
|   |-- drift_detection/
|   |-- validation/
|
|-- configs/
|   |-- model_config.yaml
|   |-- drift_thresholds.yaml
|   |-- pricing_rules_config.yaml
|
|-- tests/
|   |-- test_percentiles.py
```

```

|   |-- test_drift_metrics.py
|   |-- test_output_schema.py
|
|-- pipelines/
|   |-- run_pricing_pipeline.py
|
|-- .github/
|   |-- workflows/
|       |-- ci.yml

```

Figura 1. Estructura propuesta del repositorio para versionado y CI/CD mínimo viable.

Anexo B. Tabla propuesta model_run_log

Campo	Descripción
run_id	Identificador único de la corrida
run_timestamp	Fecha y hora de ejecución
git_commit_hash	Versión exacta del código
code_version	Versión funcional del código
config_version	Versión de configuración usada
dataset_version	Versión o snapshot del dataset
logic_version	Versión de lógica de percentiles/modelado
output_version	Versión del output generado
environment	Ambiente de ejecución
status	Estado de la corrida
executed_by	Usuario o proceso que ejecutó
comments	Comentarios o notas de ejecución

Tabla 18. Diseño inicial de la tabla model_run_log.

Anexo C. Tabla propuesta model_drift_log

Campo	Descripción
drift_id	Identificador del registro de drift
run_id	Corrida asociada
baseline_version	Baseline usado para comparación
variable_name	Variable evaluada
variable_type	Precio, cantidad u output
drift_metric	Métrica utilizada
drift_value	Valor calculado
threshold_green	Umbral aceptable
threshold_yellow	Umbral de revisión
threshold_red	Umbral crítico
drift_status	Green, yellow o red
revenue_weighted_impact	Impacto ponderado por revenue
recommended_action	No action, review, recalibrate o retrain

Tabla 8. Diseño inicial de la tabla model_drift_log.

Anexo D. Regla inicial de semáforo

Estado	Interpretación	Acción recomendada
Green	No hay cambios relevantes	Mantener modelo actual
Yellow	Hay cambios moderados	Revisar con Pricing/PM
Red	Hay cambios altos o impacto comercial relevante	Recalibrar o reentrenar

Tabla 19. Semáforo inicial para decisión de revisión o retraining.

Anexo E. Nota de confidencialidad

Debido a la naturaleza sensible de los datos utilizados en el sistema de pricing, el proyecto podrá utilizar información anonimizada, agregada, simulada o enmascarada. En caso de que la organización requiera un convenio de privacidad, este será firmado por el sponsor del proyecto y los integrantes del equipo, y se incluirá como anexo en futuras entregas.

Anexo F. Uso responsable de la cuenta de Azure

Para el desarrollo del proyecto se proporcionará una cuenta de Azure que será utilizada como ambiente de trabajo para implementar, probar y documentar la solución MLOps propuesta. Esta cuenta podrá ser utilizada para actividades como almacenamiento de archivos, ejecución de pipelines, pruebas de integración, registro de artefactos, validación de componentes y documentación técnica de la arquitectura.

El equipo deberá utilizar la cuenta con cautela, considerando que los recursos cloud pueden generar costos según el tipo de servicio, tamaño de cómputo, almacenamiento, tiempo de ejecución, transferencia de datos y configuraciones activas. Por ello, se recomienda aplicar las siguientes prácticas:

Práctica recomendada	Descripción
Monitorear consumo	Revisar periódicamente el uso y costo acumulado de los recursos creados
Apagar recursos no utilizados	Detener o eliminar servicios que no estén siendo usados
Usar recursos mínimos necesarios	Evitar máquinas, bases de datos o servicios sobredimensionados
Evitar procesos permanentes	No dejar ejecuciones activas sin supervisión
Documentar recursos creados	Mantener registro de servicios, propósito y responsable
Preferir pruebas pequeñas	Usar datasets de prueba, anonimizados o reducidos durante el desarrollo
Configurar alertas de presupuesto	Activar alertas para evitar consumir el crédito disponible
Eliminar recursos temporales	Borrar recursos de prueba una vez que ya no sean necesarios

Tabla 20. Recomendaciones para el uso responsable de la cuenta de Azure.

El objetivo es que la cuenta de Azure permita demostrar la viabilidad técnica de la capa MLOps sin comprometer el presupuesto disponible. Por lo tanto, el desarrollo deberá priorizar una arquitectura mínima viable, eficiente y controlada en costos.

Anexo G. Estructura propuesta de repositorio de infraestructura

Se propone un repositorio independiente para la gestión de infraestructura, siguiendo el enfoque de Infrastructure as Code. Para este proyecto, se considera Terraform como herramienta principal de provisionamiento, aunque también podría utilizarse Azure Bicep si se busca una integración más nativa con Azure.

De manera complementaria, se considera el uso opcional de Ansible para la configuración de recursos en caso de ser necesario. Sin embargo, para el alcance mínimo viable del proyecto, el uso de Ansible no es obligatorio, ya que muchas arquitecturas basadas en servicios gestionados pueden cubrirse completamente mediante Terraform.

La estructura propuesta del repositorio es la siguiente:

```
pricing-mlops-infra/

|-- terraform/
|   |-- main.tf
|   |-- variables.tf
|   |-- outputs.tf
|   |-- providers.tf
|   |-- backend.tf
|
|   |-- modules/
|       |-- storage_account/
|           |-- main.tf
|           |-- variables.tf
|           |-- outputs.tf
|
|       |-- compute/
|           |-- main.tf
|           |-- variables.tf
|           |-- outputs.tf
|
|       |-- key_vault/
|           |-- main.tf
|           |-- variables.tf
|           |-- outputs.tf
|
|       |-- monitoring/
|           |-- main.tf
|           |-- variables.tf
|           |-- outputs.tf
|
|   |-- environments/
|       |-- dev.tfvars
|       |-- staging.tfvars
|       |-- prod.tfvars
|
|-- ansible/
|   |-- inventories/
|       |-- dev.ini
|       |-- staging.ini
|
|   |-- playbooks/
|       |-- setup_compute.yml
|       |-- configure_environment.yml
```

```
|-- pipelines/
|   |-- infra-ci.yml
|   |-- infra-cd.yml

|-- scripts/
|   |-- init.sh
|   |-- plan.sh
|   |-- deploy.sh
|   |-- destroy-temp.sh

|-- docs/
|   |-- architecture.md
|   |-- environments.md
|   |-- cost-management.md
|   |-- security-guidelines.md

|-- README.md
```

Figura 2. Estructura propuesta del repositorio de infraestructura para IaC y CI/CD.

Anexo H. Convenio de confidencialidad firmado

En este apartado se incluye el convenio de confidencialidad firmado por el sponsor del proyecto y los integrantes del equipo, el cual autoriza el uso de la información para fines académicos y establece las condiciones de protección de los datos utilizados.



COMPROMISOS DE COLABORACIÓN PARA EL PROYECTO INTEGRADOR

Escuela de Ingeniería y Ciencias

Maestría en Inteligencia Artificial Aplicada

MNA

Figura H.1. Convenio de confidencialidad firmado (página 1 de 4).

GENERALES

El *Proyecto Integrador*, corresponde al proyecto final que se realiza del programa de la Maestría en Inteligencia Artificial Aplicada. Éste se desarrollará y evaluará en la materia con clave TC5035, del mismo nombre.

El objetivo principal del proyecto es resolver una problemática real, vigente, sea del sector industrial o de instituciones gubernamentales, ONG u algún otro tipo de institución. Lo anterior, previa autorización del comité.

Para efectos del presente documento el programa de Maestría en Inteligencia Artificial Aplicada (en adelante MNA) y YAGEO (en adelante el cliente) reconocen los siguientes compromisos:

COMPROMISOS

Para realizar el proyecto integrador la MNA se compromete a lo siguiente:

1. Asignar un asesor de proyecto con experiencia y vinculación en la problemática presentada por el cliente.
2. Ofrecer las instalaciones y equipo existente requerido que contribuya a resolver la problemática identificada.
3. Proporcionar retroalimentación constante y sistemática durante el registro de la propuesta y desarrollo del proyecto en la materia TC5035.
4. Conformar el comité de evaluación una vez que se hayan alcanzado los objetivos presentados en la propuesta del Proyecto Integrador.

El cliente se compromete a:

1. Asignar a un patrocinador con reconocidas fortalezas en el área de dominio a atender por el proyecto.
2. Retroalimentar a(los) integrante(s) del proyecto con relación a los resultados parciales obtenidos.
3. Asegurar los recursos de información requeridos para la realización del proyecto.
4. Reconocer la participación del estudiante o los estudiantes, así como al Tecnológico de Monterrey, en las publicaciones o reportes que se generen del proyecto.

Ambas instituciones se comprometen a:

1. Guardar la confidencialidad con relación a los resultados obtenidos que sean sensibles/estratégicos para cualquiera de las instituciones.

Figura H.2. Convenio de confidencialidad firmado (página 2 de 4).

2. Interacción bidireccional en reuniones de trabajo para optimizar el proceso de implementación del Proyecto Integrador.

La colaboración se realizará, (sin limitarse a ello) para la implementación del Proyecto de Y-PIE MLOPS a partir del 20 de abril y concluyendo tentativamente el 3 de julio del 2026.

Alumno(s) de Proyecto Integrador

Emanuel Flores Martínez – A01796497



David Alberto Serrano Garcia – A01795935



Mario Javier Soriano Aguilera – A01384282



Nombre, matrícula y firma

Por la MNA

Dr. Luis Eduardo Falcón. Director Nacional de la Maestría en Inteligencia Artificial Aplicada

Nombre, puesto y Firma

Patrocinador asignado por el cliente

Figura H.3. Convenio de confidencialidad firmado (página 3 de 4).

A handwritten signature in black ink, appearing to be 'CV', is written over a horizontal line.

Carlos Vidales, Director de Operaciones de Venta
Nombre, puesto y Firma

Figura H.4. Convenio de confidencialidad firmado (página 4 de 4).